Queues: First In, First Out

Programming and Data Structures — Week 3, Lecture 2

Dr. Innocent Nyalala

Objectives

By the end of this lecture, you will be able to explain First In, First Out (FIFO) ordering and contrast it directly against the Last In, First Out ordering from the previous lecture; implement a naive array-backed `Queue` class and identify precisely where it hides an O(n) cost that is easy to miss on first inspection; explain why that cost exists, using Week 2 Lecture 4's shifting analysis rather than a new unexplained rule; and state what a genuinely correct queue implementation needs to do differently to avoid it.

Outline

Today covers a cafeteria-line analogy for FIFO ordering; a precise definition of FIFO, contrasted directly against the previous lecture's LIFO; a naive array-backed queue implementation; the hidden O(n) cost this naive implementation carries, traced in full; why this cost matters at real scale, backed by measurement; the fix, using both ends of a `deque`; a worked application, a print job queue; an in-class quiz; and a summary.

The cafeteria-line analogy

Picture a cafeteria line: every new person joins at the back, and the person being served leaves from the front. The first person to join the line is guaranteed to be the first person served, and nobody already in line can be served before someone who joined earlier — a direct contrast to the plate stack from the previous lecture, where the most recently added item was served first. This mirrored relationship between the two structures is worth holding onto for the rest of this lecture.

Defining FIFO precisely, against LIFO

A queue supports two core operations: `enqueue(x)`, which adds `x` to the back, and `dequeue()`, which removes and returns whatever is at the front. This is the exact structural mirror of last lecture's stack, which added and removed from the *same* end (the top); a queue deliberately adds and removes from *opposite* ends, which is precisely what produces First In, First Out ordering instead of Last In, First Out.

Other everyday FIFO examples

FIFO ordering shows up constantly outside the cafeteria: a printer queue prints the first document sent to it before later ones, a numbered-ticket system at a bank or clinic calls the earliest-issued number first, and a customer support system generally prioritizes the oldest unresolved ticket. In every case, the defining property is that order of arrival determines order of service.

The naive array-backed queue

```
class NaiveQueue:
    def __init__(self):
        self._items = []

    def enqueue(self, x):
        self._items.append(x)      # add to the back - Week 2: O(1) amortized

    def dequeue(self):
        if not self._items:
            raise IndexError("dequeue from an empty queue")
        return self._items.pop(0)   # remove from the FRONT - index 0
```

This is the most direct translation of the queue definition into code: `enqueue` appends to the back of the underlying list, and `dequeue` removes from the front using `pop(0)`. The `enqueue` side looks entirely unremarkable — appending to the end is exactly the same O(1) amortized operation the stack used for `push`. The trap is hiding in `dequeue`, and it is easy to miss on a first read, because `pop(0)` reads as "just remove the first element" without any obvious warning sign in the syntax itself.

The hidden trap, revealed

`pop(0)` removes the element at index 0 of the underlying array — this is exactly the start-deletion case analyzed in Week 2 Lecture 4, identified there as the single most expensive position for any array

operation. Every remaining element in the array must shift one position to the left to close the gap left at the front. The cost of `dequeue()` is therefore O(n), not O(1) — and critically, this happens on *every single call*, not merely occasionally the way a dynamic array's resize does. A queue built this way is silently O(n) per dequeue, for its entire lifetime.

Tracing the shift, element by element

```
Before dequeue(): [10, 20, 30, 40, 50]   (front is index 0: value 10)

Step 1: move index 1 (20) → index 0:  [20, 20, 30, 40, 50]
Step 2: move index 2 (30) → index 1:  [20, 30, 30, 40, 50]
Step 3: move index 3 (40) → index 2:  [20, 30, 40, 40, 50]
Step 4: move index 4 (50) → index 3:  [20, 30, 40, 50, 50]
Step 5: shrink length by 1:           [20, 30, 40, 50]
```

Tracing `dequeue()` on a 5-element queue makes the cost concrete: removing the front element (10) requires shifting all 4 remaining elements one position left, one at a time, before the array's reported length shrinks. This is not a one-time cost paid occasionally, unlike a dynamic array's resize — it happens identically on *every* call to `dequeue()`, for as long as the queue has more than a handful of elements remaining.

Why this matters at scale

Consider a print queue that processes 10,000 jobs over its lifetime, one `dequeue()` call per job. Each dequeue shifts however many jobs remain, so the total shifting work across all 10,000 dequeues is roughly $1 + 2 + \cdots + 10{,}000 \approx 50{,}000{,}000$ element-moves — this is precisely the same O(n²) shape of cost that Week 2 identified for a "grow by one slot" dynamic array strategy. It is worth explicitly noticing this recurrence: it is the *same underlying mistake* — repeatedly paying an O(n) cost for an operation that should be O(1) — appearing in a different data structure. Recognizing this shape is more valuable than memorizing either specific example.

Measuring it for real

```python
import time

def time_naive_queue(n):
    q = []
    for i in range(n):
        q.append(i)
```

```
    start = time.perf_counter()
    for i in range(n):
        q.pop(0)
    return time.perf_counter() - start

for n in (2_000, 4_000, 8_000):
    print(n, time_naive_queue(n))
```

Running this measurement for growing values of n should show the total `dequeue` time growing faster than linearly in n — roughly quadrupling when n doubles, the signature of an $O(n^2)$ total cost. This is the same style of direct measurement used in Week 2 to confirm amortized append cost; the habit of testing a complexity claim rather than only reasoning about it abstractly applies here identically.

Fixing it: `collections.deque`

```
from collections import deque

class Queue:
    def __init__(self):
        self._items = deque()

    def enqueue(self, x):
        self._items.append(x)        # add to the back - O(1)

    def dequeue(self):
        if not self._items:
            raise IndexError("dequeue from an empty queue")
        return self._items.popleft()  # remove from the front - O(1)
```

The fix is exactly the structure introduced as an MTech addition in Week 2 Lecture 4: Python's `collections.deque`, implemented internally as a doubly linked list of fixed-size blocks rather than one contiguous array. `popleft()` requires no shifting whatsoever — removing from the front is handled by adjusting an internal pointer to what now counts as the front, not by physically moving every remaining element. Both `enqueue` (via `append`) and `dequeue` (via `popleft`) are genuinely O(1), with no hidden per-call cost — the trap from the naive implementation is fully closed.

Side-by-side: naive queue vs. `deque`-backed queue

This table is the entire lecture in one line: the naive array-backed queue and the `deque`-backed queue look nearly identical in code, differ by one data structure choice, and differ by an entire complexity

class on `dequeue`. This is a strong argument for always asking, of any structure, "what does the *other* end cost?" — a structure that looks efficient at a glance can be hiding an expensive operation one line away.

Application: a print job queue

```
jobs = Queue()
jobs.enqueue("report.pdf")
jobs.enqueue("photo.png")
jobs.enqueue("invoice.pdf")

while True:
    try:
        current = jobs.dequeue()
    except IndexError:
        break
    print("printing:", current)
```

This print job queue enqueues three documents in submission order and then processes them one at a time via `dequeue`, printing `report.pdf` first (the first submitted), then `photo.png`, then `invoice.pdf` — FIFO order preserved exactly as submitted. A real print spooler is more elaborate (priority levels, cancellation), but the core ordering guarantee is precisely this queue's `enqueue`/`dequeue` behavior.

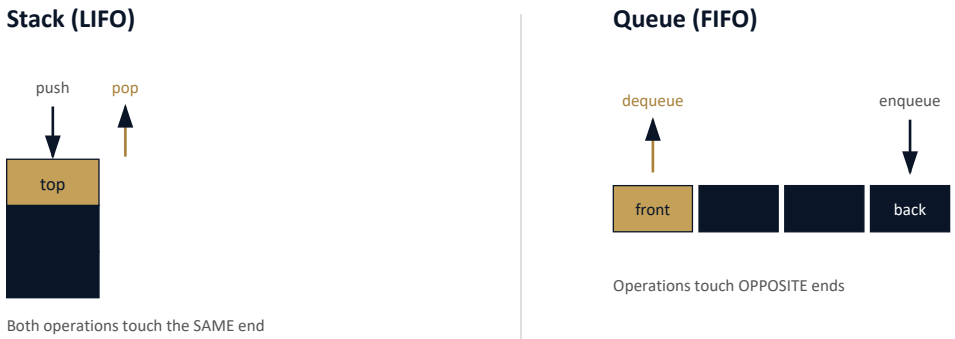
Tracing the print queue

```
enqueue("report.pdf")  → queue: [report.pdf]
enqueue("photo.png")   → queue: [report.pdf, photo.png]
enqueue("invoice.pdf") → queue: [report.pdf, photo.png, invoice.pdf]

dequeue() → "report.pdf"  → queue: [photo.png, invoice.pdf]
dequeue() → "photo.png"   → queue: [invoice.pdf]
dequeue() → "invoice.pdf" → queue: []
dequeue() → IndexError (empty)
```

Tracing the full lifecycle: three enqueues build the queue in submission order, and three dequeues drain it in the exact same order — the first document submitted is the first one printed, and the process ends with an `IndexError` once the queue is empty, exactly matching the earlier definition of `dequeue` on an empty queue.

The picture: two ends, two operations



This single structural difference in where operations are allowed to touch is what separates LIFO from FIFO.

Placing the stack and queue side by side makes the structural difference visually immediate: a stack's two operations act on the *same* end, while a queue's two operations act on *opposite* ends — this single difference in where operations are allowed to touch is what separates LIFO from FIFO.

MTech addition — implementing a queue with two stacks

```
class TwoStackQueue:
    def __init__(self):
        self._in_stack = []
        self._out_stack = []

    def enqueue(self, x):
        self._in_stack.append(x)

    def dequeue(self):
        if not self._out_stack:
            while self._in_stack:
                self._out_stack.append(self._in_stack.pop())
        if not self._out_stack:
            raise IndexError("dequeue from an empty queue")
        return self._out_stack.pop()
```

A queue can also be built from two stacks, a construction worth knowing both as a common interview question and as another worked example of amortized analysis. `enqueue` simply pushes onto `_in_stack`. `dequeue` checks `_out_stack` first; if it is empty, every element is popped off `_in_stack` and pushed onto `_out_stack`, which reverses their order once — since `_in_stack` held

elements in LIFO order, reversing them once restores FIFO order in `_out_stack`. Only then is the top of `_out_stack` popped and returned.

MTech addition — why this is still amortized O(1)

Although a single `dequeue` call that triggers the "flip" can cost O(n), each individual element is moved from `_in_stack` to `_out_stack` at most once over the structure's entire lifetime — once moved, it stays in `_out_stack` until popped and is never moved back. Summed over any sequence of n enqueue and dequeue operations, the total work is therefore O(n), not O(n²), making the amortized cost per operation O(1) — the exact same style of argument as Week 2 Lecture 3's coin-jar and potential-function proofs for dynamic array append, applied here to a different structure.

Exercise

As an exercise, implement a `peek()` method for `TwoStackQueue` that returns the front element without removing it, being careful to still perform the "flip" from `_in_stack` to `_out_stack` when necessary. Before running any code, trace by hand the sequence `enqueue(1); enqueue(2); enqueue(3); dequeue(); enqueue(4); dequeue()`, recording the exact contents of both `_in_stack` and `_out_stack` after each call.

Quiz — spot the cost

Given a queue implemented with a plain Python list, where `enqueue` uses `append` and `dequeue` uses `pop(0)`, work out the total cost of processing all n items — each enqueued once and dequeued once — before checking the answer.

Quiz — answer

The n `enqueue` calls cost O(n) total, since each is O(1) amortized — no surprise there. The n `dequeue` calls, however, cost O(n²) total: the first dequeue shifts up to $n - 1$ elements, the next shifts up to $n - 2$, and so on, summing to roughly $n^2/2$ total element-moves. The overall cost of processing the queue is therefore O(n²), entirely dominated by the naive `dequeue` implementation — precisely the trap this lecture set out to expose, and precisely what switching to `deque` or the two-stack construction avoids.

Summary

A queue enforces First In, First Out ordering by adding at the back and removing from the front. The naive array-backed implementation hides an O(n) `dequeue`, a direct instance of the start-deletion trap identified in Week 2 Lecture 4 — the same mistake, recurring in a new structure. `collections.deque` fixes this by using a fundamentally different internal layout, making both `enqueue` and `dequeue` genuinely O(1). A two-stack queue achieves the same amortized O(1) bound by a completely different route, moving each element between two stacks at most once over its lifetime. Next lecture leaves arrays behind entirely: linked lists remove the contiguous-memory constraint altogether, trading array's O(1) index access for O(1) insertion anywhere, once you already hold a reference to the right place.